# Fundamentación Teórica Extendida: Unidad 5 – Clase 2

## *Amenazas críticas a la base de datos y gobierno/protección de datos personales*

**Cátedra:** Base de Datos II – Tecnicatura Universitaria en Programación (UTN-FRM)

**Marco de referencia:** PostgreSQL 16+, OWASP Top 10, Ley Nacional 25.326 y estándares RGPD/GDPR.

### 1. Introducción al Enfoque de Seguridad Ofensiva y Defensiva

La administración de bases de datos relacionales no concluye con la definición declarativa de roles y privilegios (Clase 1). En entornos reales, los sistemas operan bajo una premisa ineludible: **los modelos de autorización fallan o son objeto de ataques deliberados**.

La seguridad de la base de datos debe concebirse bajo el paradigma de **defensa en profundidad** (*defense-in-depth*), donde coexisten múltiples barreras ortogonales e independientes. Si un atacante burla la capa de presentación o la lógica de negocio en la aplicación, el motor de la base de datos debe actuar como la última línea de contención estricta mediante tipado, parametrización, privilegios mínimos y aislamiento.

### 2. Inyección de SQL (SQL Injection - SQLi)

#### 2.1. Naturaleza y Mecánica del Ataque

La inyección de SQL surge fundamentalmente de la **confusión entre código y datos** en el nivel del intérprete o analizador sintáctico (*parser*) del motor. Ocurre cuando una aplicación concatena entradas externas no confiables dentro del cuerpo léxico de una sentencia SQL antes de enviarla a compilar y planificar. Al interpretar la cadena resultante, el motor evalúa los caracteres de control introducidos por el atacante (comillas, delimitadores, guiones de comentario) alterando el Árbol de Sintaxis Abstracta (*AST*) de la consulta.

**Análisis del Vector Vulnerable**

Supóngase una implementación defectuosa de autenticación dinámica:

```
consulta := 'SELECT id, rol FROM usuario WHERE mail = '''
        || p_mail || ''' AND contrasena = ''' || p_pass || ''';';
EXECUTE consulta INTO v_id, v_rol;
```

Si el usuario envía como entrada para p_mail:

Plaintext

```
' OR '1'='1' --
```

La sentencia resultante ensamblada en el buffer del motor es

```
SELECT id, rol FROM usuario WHERE mail = '' OR '1'='1' --' AND contrasena = '...';
```

- **Alteración del predicado lógico:** El operador booleano OR '1'='1' transforma la cláusula WHERE en una tautología (siempre verdadera para cualquier tupla).
- **Truncamiento de la validación:** La secuencia -- comenta el resto de la instrucción, descartando de plano la verificación de la contraseña.
- **Impacto colateral:** La sentencia devuelve la primera tupla procesada del escaneo secuencial. Si la primera fila corresponde al administrador de la plataforma, se produce una vulneración total de acceso sin necesidad de conocer credenciales.

### 2.2. Mitigación Rigurosa: Parametrización y Sentencias Preparadas (*Prepared Statements*)

La única defensa estructural y definitiva frente a SQLi es **separar el flujo de control del flujo de datos**. Técnicamente, esto se implementa mediante la parametrización de consultas:

1. **Fase de Análisis/Compilación:** El motor analiza la plantilla SQL con marcadores posicionales (p. ej., $1, $2 o variables tipadas en PL/pgSQL), genera el plan de ejecución y fija la estructura del comando de forma inmutable.
2. **Fase de Vinculación (*Binding*):** Los valores de los parámetros se transmiten al motor en paquetes binarios independientes. El analizador sintáctico nunca vuelve a ejecutarse sobre estos valores; se tratan exclusivamente como literales.

```
-- En PL/pgSQL nativo (los identificadores de parámetros viajan tipados)
SELECT id, rol INTO v_id, v_rol
FROM usuario
WHERE mail = p_mail
  AND contrasena = crypt(p_pass, contrasena);


-- En SQL dinámico mediante EXECUTE ... USING
EXECUTE 'SELECT id, nombre FROM producto WHERE categoria_id = $1'
INTO v_resultado
USING p_categoria_id;
```

*Nota técnica:* Intentar mitigar la inyección filtrando comillas o sustituyendo caracteres manualmente es considerado un antipatrón en la industria, ya que los atacantes pueden eludir listas negras mediante codificaciones complejas o caracteres nulos.

### 2.3. Defensa en Profundidad: El Rol de la Conexión de Aplicación

Aunque ocurra una inyección por error en la capa de software, el principio de **mínimo privilegio** limita el radio de daño (*blast radius*). Si la conexión de la aplicación web (app_web) carece de permisos de DDL (DROP, ALTER), de TRUNCATE o de DELETE global

sobre tablas maestras, una inyección que intente destruir el esquema fallará de inmediato con un error de permisos.

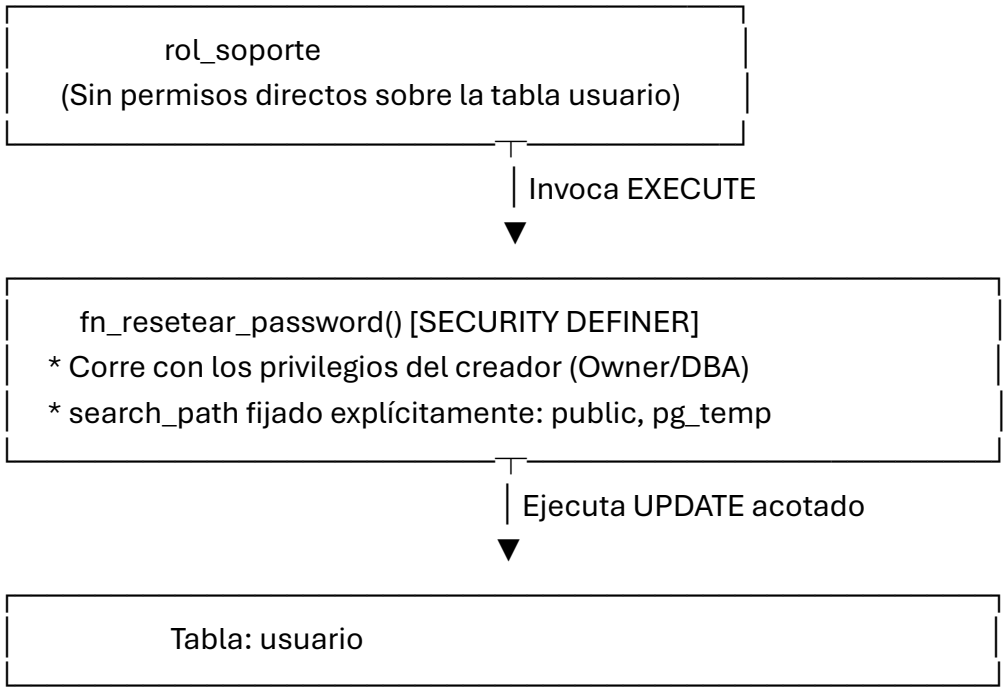
### 3. Escalamiento de Privilegios

#### 3.1. Tipología

- **Escalamiento Vertical:** Un sujeto o rol con bajos privilegios (ej. rol_soporte o un usuario final) adquiere capacidades de superusuario o de administrador de base de datos (ADMIN, SUPERUSER).
- **Escalamiento Horizontal:** Un sujeto accede indebidamente a objetos, registros o funciones asignadas a otra identidad que comparte su mismo nivel de privilegios (ej. el cliente A consultando pedidos del cliente B).

#### 3.2. Vulnerabilidades en Entornos PostgreSQL y Funciones SECURITY DEFINER

En PostgreSQL, una función se define de forma predeterminada como SECURITY INVOKER, ejecutándose bajo los permisos del usuario que la llama. No obstante, muchas operaciones legítimas (como permitir al personal de soporte resetear contraseñas de clientes sin darles acceso UPDATE directo a toda la tabla usuario) exigen ejecutarse con privilegios elevados. Para ello se utiliza SECURITY DEFINER.




**El Vector del** search_path **Vulnerable (Ataque de Suplantación de Esquema)**

Si una función SECURITY DEFINER no fija de forma estricta su ruta de búsqueda (search_path), el motor intentará resolver los nombres de las tablas y funciones sin calificar según la prioridad del usuario que invoca:

1. Un atacante crea un esquema propio o explota un esquema con permisos de creación (como históricamente ocurría en public).
2. Define una tabla o función con el mismo nombre que la utilizada internamente por la rutina privilegiada.
3. Al llamar a la función SECURITY DEFINER, esta resuelve e interactúa con el objeto malicioso bajo los privilegios del administrador.

**Construcción segura y hardening:**

```
CREATE OR REPLACE FUNCTION fn_resetear_password(
   p_usuario_id BIGINT,
   p_nueva TEXT
) RETURNS VOID
SECURITY DEFINER
-- Mitigación obligatoria: fija los esquemas de resolución de nombres
SET search_path = public, pg_temp
AS $$
BEGIN
   UPDATE usuario
   SET contrasena = crypt(p_nueva, gen_salt('bf'))
   WHERE id = p_usuario_id AND eliminado = FALSE;
END;
$$ LANGUAGE plpgsql;


-- Mitigación de permisos por omisión

REVOKE ALL ON FUNCTION fn_resetear_password(BIGINT, TEXT) FROM PUBLIC;

GRANT EXECUTE ON FUNCTION fn_resetear_password(BIGINT, TEXT) TO
rol_soporte;
```

- SET search_path = public, pg_temp**:** Neutraliza la inyección de esquemas al obligar al motor a buscar únicamente en public y desestimar objetos temporales manipulados fuera de su propio contexto.
- REVOKE … FROM PUBLIC**:** Elimina el permiso de ejecución implícito que PostgreSQL asigna a todo el mundo por defecto.

**4. Fuga de Información (*Data Leakage*)**

**4.1. Vectores Críticos en Sistemas de Bases de Datos**

1. **Copias de Seguridad (Backups) sin cifrar:** El respaldo almacena el estado completo de la base. Si se transporta a través de la red o se almacena en medios

secundarios sin cifrado en reposo, anula todas las protecciones implementadas dentro del motor.

2. **Exposición en Vistas por uso de** SELECT ***:** Si una vista pública se define con SELECT *, cualquier columna confidencial que se agregue en el futuro a la tabla subyacente (ej. número de teléfono o datos internos) quedará expuesta de forma inadvertida. Se exige listar explícitamente las columnas necesarias (*proyección estricta*).
3. **Mensajes de Error Verborrágicos:** Los volcados de excepciones de base de datos dirigidos hacia interfaces web revelan tablas, columnas y tipos de datos. Deben atraparse a nivel aplicación devolviendo errores opacos al usuario mientras el detalle se confina al log protegido del servidor.
4. **Logs Transaccionales con Parámetros Sensibles:** Los registros del motor (log_statement = 'all') pueden capturar sentencias completas con datos sensibles o contraseñas en texto plano.
5. **Ataques por Canales Laterales (*Timing Attacks*):** Diferencias de tiempo de respuesta al validar credenciales permiten determinar si un usuario existe o no antes de validar la clave.

## 5. Gobierno y Protección de Datos Personales

### 5.1. Marco Jurídico y Principios Rectores (Ley 25.326 y RGPD)

Los administradores de bases de datos tienen la responsabilidad legal y ética sobre la información sensible que gestionan. Los principios esenciales aplicables al modelado y arquitectura son:

- **Minimización de Datos:** Solo se deben recolectar y persistir aquellos atributos estrictamente necesarios para cumplir con la finalidad del sistema. Si un dato no existe en la base, no puede ser sustraído ni filtrado.
- **Limitación de la Finalidad:** Los datos recopilados no pueden utilizarse para fines distintos de aquellos comunicados y autorizados por el titular.
- **Limitación del Plazo de Conservación:** La información personal debe conservarse únicamente durante el tiempo exigido por las necesidades del negocio o la regulación fiscal/legal.

### 5.2. Técnicas de Desvinculación de Datos

| Mecanismo | ¿Es Reversible? | Impacto Metodológico / Técnico | Caso de Uso Típico |
|---|---|---|---|
| **Anonimización** | **No** (Irreversible) | Destruye definitivamente el nexo causal entre el dato y el sujeto. El dato deja de ser formalmente un dato personal. | Ambientes de prueba (QA), analítica masiva y consultas a asistentes de IA. |
| **Seudonimización / Enmascaramiento** | **Sí** (Reversible con clave o mapeo externo) | Sustituye los atributos identificadores directos por un valor artificial o token. Requiere salvaguardar la tabla/clave de correspondencia de forma aislada. | Cumplimiento regulatorio en entornos donde la reversión es legalmente necesaria. |
| **Hashing con Sal** | **No** (Unidireccional) | Genera una síntesis criptográfica fija e irreversible a partir de una función resumen robusta (ej. Blowfish, Argon2). | Almacenamiento seguro de credenciales (contrasena). |
| **Generalización** | **No** (Pérdida de precisión) | Reduce la granularidad del atributo para evitar la identificación unívoca (ej. reemplazar fecha exacta por grupo de edad). | Reportes estadísticos y tableros de BI. |

### 5.3. Estrategia de Enmascaramiento Estático para Entornos No Productivos

Para permitir el desarrollo de software, pruebas de rendimiento y análisis sin violar la privacidad, se construyen datasets anonimizados deterministas:

```
CREATE TABLE usuario_anon AS
SELECT
   id,
   'Usuario ' || id AS nombre,
   'Apellido ' || id AS apellido,
   'usuario' || id || '@ejemplo.test' AS mail,
   NULL::TEXT AS celular,
   'hash_no_valido' AS contrasena,
   rol,
   eliminado,
   created_at
FROM usuario;
```

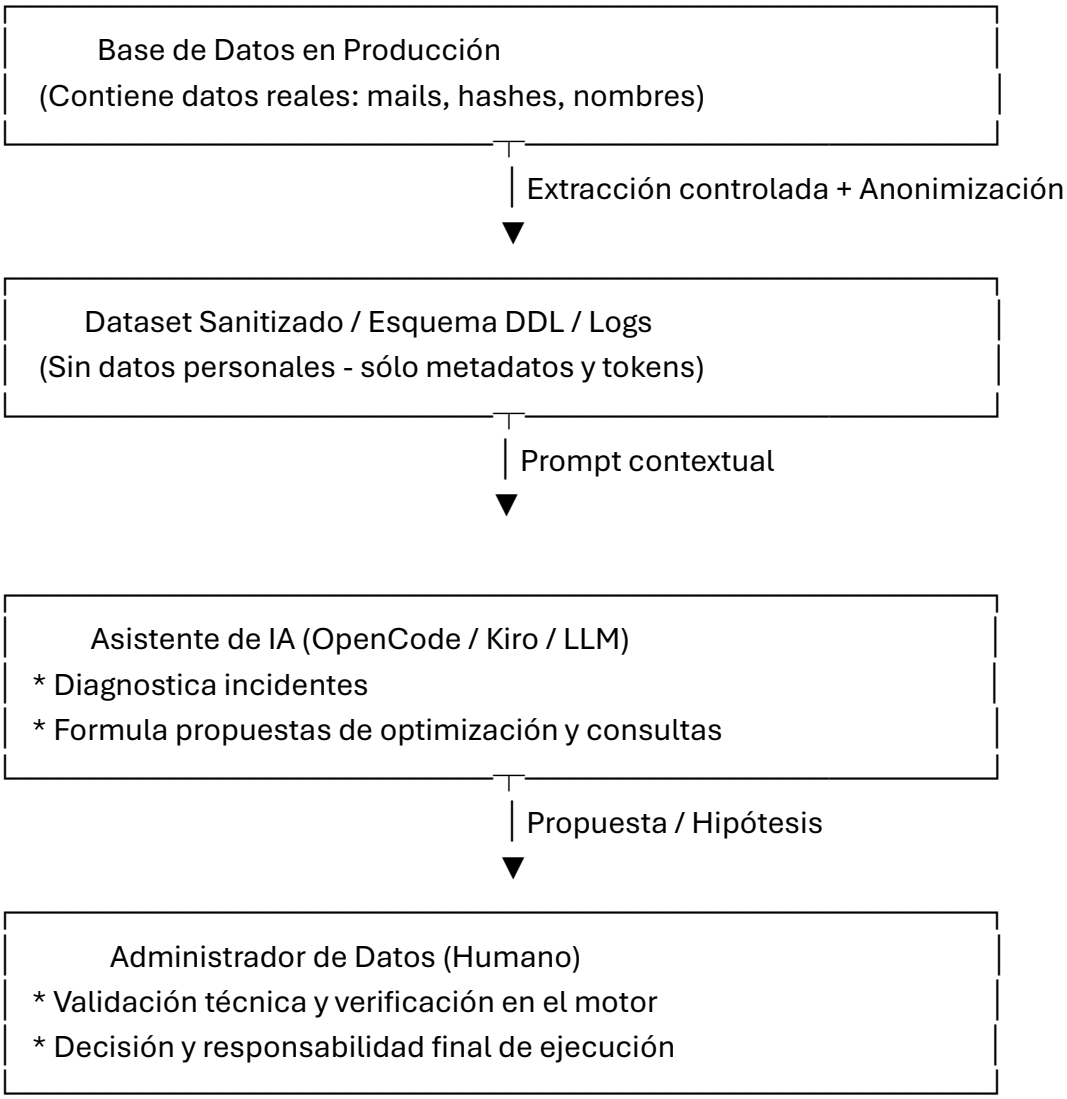
- **Determinismo de pruebas:** Usar 'Usuario ' || id y correos sintéticos correlativos preserva la cardinalidad de la tabla y la selectividad de los índices.
- **Sanitización de credenciales:** El valor literal 'hash_no_valido' garantiza que las cuentas no puedan autenticarse bajo ninguna combinación posible en entornos de testing.

### 6. La Frontera de la Privacidad con la Inteligencia Artificial

Dentro del marco formativo de la carrera, la IA opera como un motor primario mediador, lo que impone delimitar de forma estricta los datos que pueden interactuar con modelos externos (LLMs o agentes de codificación como OpenCode y Kiro):

### 6.1. Reglas de Contención de Datos

1. **Prohibición Absoluta de Datos Reales en Prompts:** Ningún dato personal (nombres, teléfonos, correos, documentos, contraseñas) ni respaldos de producción se envían a motores de IA.
2. **Uso Exclusivo de Esquemas y Metadatos:** La IA debe recibir definiciones de DDL (CREATE TABLE, índices), planes de ejecución textuales (EXPLAIN ANALYZE) y datasets sintéticos o anonimizados.
3. **Sanitización de Registros y Logs de Auditoría:** Si se somete un log transaccional para un simulacro o diagnóstico de incidentes, las direcciones IP internas y los identificadores personales deben ser tokenizados previamente.




### 6.2. Protocolo de Validación Humana

La IA acelera la identificación de patrones (ej. detectar un ataque de fuerza bruta a partir de cientos de intentos de inicio de sesión seguidos de lecturas masivas). Sin embargo, **la narrativa de la IA es únicamente una hipótesis técnica que requiere contrastación empírica en el motor**:

- Se contrasta el incidente contra vistas estadísticas del sistema (pg_stat_activity, logs del sistema operativo).
- Se aíslan los rangos de direcciones IP y se comprueba la consistencia de los respaldos.
- **La responsabilidad última es indelegable:** Ningún script de contención o remediación sugerido por IA se ejecuta en el servidor sin previa lectura línea por línea, prueba en copia descartable y ejecución en transacción controlada.